

Lecture 1: Computation and Abstraction

Dr. Changlai Du

Learning goals

After this lecture, you should be able to explain what an algorithm is, distinguish a function from an algorithm, describe the Turing-machine model and the universal Turing machine, state the Church–Turing thesis, compare algorithms through time and space complexity, and explain why hierarchical abstraction makes modern computing systems manageable.

Contents

1	Computation as a Mechanical Process	4
1.1	A motivating problem	4
1.2	Algorithm, input, and output	5
1.3	Correctness and efficiency	6
2	From a Human Calculator to a Turing Machine	7
2.1	The abstract machine	7
2.2	Why one-dimensional and unbounded?	7
2.3	Finite-state control	7
2.4	Unbounded execution and halting	8
2.5	A Turing-machine view of long division	9
3	More Algorithms and More Machines	10
3.1	The Euclidean algorithm	10
3.2	The prime-factorization method	10
3.3	Functions versus algorithms	11
4	The Universal Turing Machine	12
5	The Church–Turing Thesis	13
5.1	What would challenge the thesis?	13
6	Limits of Computation	14
6.1	Computable numbers	14
6.2	The halting problem	14
7	Stored-Program Computers	15
8	Complexity: The Cost of Computation	16
8.1	Input size and asymptotic growth	16
8.2	Searching an attendance list	16
8.3	Fibonacci numbers	16
8.4	P and NP	17
9	Hierarchical Abstraction	18
9.1	Leaky abstractions	18
10	Concept Checklist	19

11 Study Questions	20
Homework	21
Sources Used for These Notes	22

1 Computation as a Mechanical Process

1.1 A motivating problem

Consider the decision problem

Is 35,500,001 divisible by 113?

Long division gives

$$35,500,001 = 113 \times 314,159 + 34.$$

The remainder is 34, so the answer is *no*.

The familiar long-division procedure consists of a small collection of steps:

1. divide or estimate the next quotient digit;
2. multiply the divisor by that digit;
3. subtract;
4. bring down the next input digit;
5. repeat until no input remains.

Nothing in these steps requires a new creative insight once the procedure has been specified. A person or a machine can follow the same rules. This repeatable, rule-governed character is the first sense in which computation is *mechanical*.

$$\begin{array}{r}
 314159 \\
 113 \overline{) 35500001} \\
 \underline{339} \\
 160 \\
 \underline{113} \\
 470 \\
 \underline{452} \\
 180 \\
 \underline{113} \\
 670 \\
 \underline{565} \\
 1051 \\
 \underline{1017} \\
 34
 \end{array}$$

In treating long division as a mechanical process, we assume that its elementary operations—such as short division, single-digit multiplication, subtraction, comparison, and bringing down the next digit—can themselves be carried out mechanically. We call these primitive operations *instructions*; the finite collection of instructions available to a machine is its *instruction set*. At this level of abstraction, we specify what each instruction does without yet explaining how the machine implements it.

The implementation of such instructions is studied at lower levels of computer systems. Relevant topics and courses include Boolean algebra and switching theory, digital logic design, combinational and sequential circuits, arithmetic circuits such as adders and multipliers, computer

organization and architecture, instruction-set architecture, and processor microarchitecture. These subjects show how elementary mechanical steps can ultimately be realized using logic gates, registers, data paths, and control circuits.

1.2 Algorithm, input, and output

Algorithm

An *algorithm* is a finite, unambiguous sequence of instructions for solving a class of problems. It specifies how to transform an input into an output.

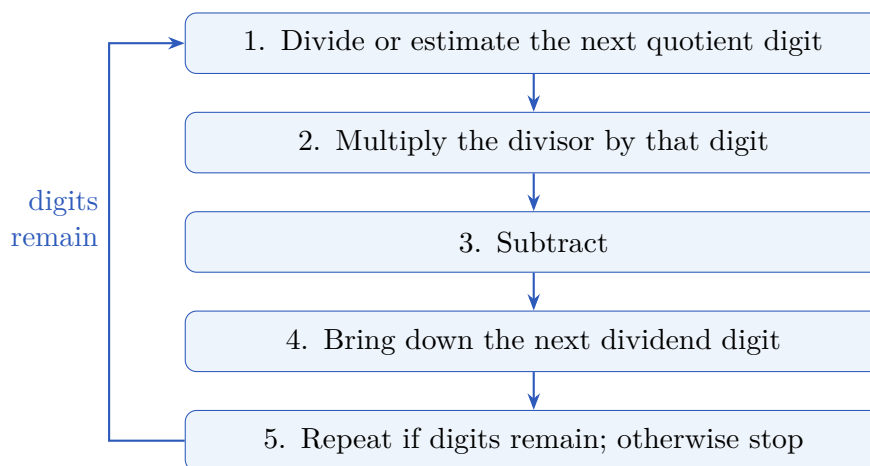


Figure 1: The five-step long-division algorithm.

For the divisibility example, an input may consist of a dividend a and a divisor b . Although our numerical example is small enough to write on one line, computer science studies the entire family of possible inputs rather than only this one instance. The dividend may be extremely long—possibly millions or billions of digits—so an algorithm must describe a uniform procedure that works regardless of the particular input length. If the dividend has n digits, we treat n as its input size and ask how the algorithm’s time and memory usage grow as n increases. The requested output then determines the type of problem:

Problem type	Required output	Example
Decision	yes or no	Is a divisible by b ?
Search	a particular object or value	What is the remainder of a/b ?
Counting	the number of valid objects	How many divisors does a have?
Optimization	the best feasible object or value	Which valid solution has minimum cost?

A search problem asks us to produce a particular object, often called a *solution* or *witness*, that satisfies specified conditions. For many important search problems, if we are given a candidate solution, we can verify efficiently that it is correct, even though finding such a solution may be much harder. This distinction between finding and verifying a solution will become important when we discuss the complexity class NP.

The output 34 solves the remainder-search problem. The output *no* solves the original decision problem.

1.3 Correctness and efficiency

The same function can have several algorithms. Repeatedly subtracting 113 from 35,500,001 will eventually produce the correct remainder, but it needs roughly 314,159 subtractions. Long division processes the input one digit at a time and therefore requires a number of rounds proportional to the number of digits.

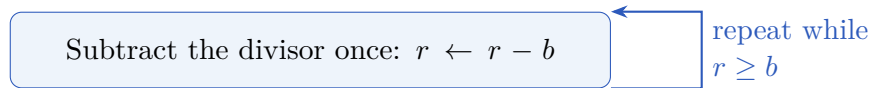


Figure 2: The repeated-subtraction algorithm for the divisibility problem.

This example separates two questions:

- **Computability:** Can any mechanical procedure solve the problem for every valid input?
- **Complexity:** How much time and memory does a particular procedure require as the input grows?

2 From a Human Calculator to a Turing Machine

When performing long division by hand, we use paper as memory, symbols for data, eyes to read, a hand and pencil to write, a finite collection of remembered instructions, and rules that determine the next action. Turing's model keeps only these essential components.

2.1 The abstract machine

A Turing machine has:

- an unbounded one-dimensional tape divided into cells;
- a finite tape alphabet, including a blank symbol;
- a read/write head that scans one cell at a time;
- a finite set of control states, including a start state and one or more halting states;
- a transition function that determines the next action.

The transition function has the form

$$\delta(q, a) = (q', b, D),$$

where q is the current state, a is the scanned symbol, q' is the next state, b is the symbol written to the tape, and $D \in \{L, R\}$ is the direction of movement.

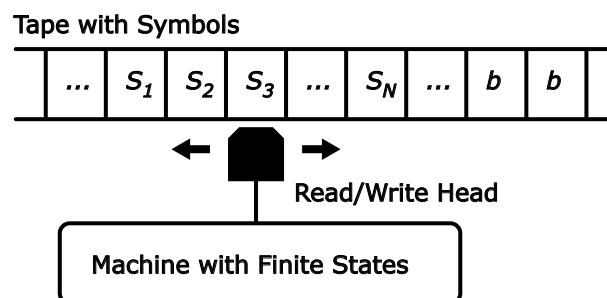


Figure 3: The basic components of a Turing machine.

2.2 Why one-dimensional and unbounded?

A sheet of paper is two-dimensional, but its cells can be enumerated and placed along a one-dimensional tape. This transformation simplifies the mechanism: the head needs only to move left or right. The precise enumeration is unimportant as long as it preserves all information.

The tape is *unbounded* because the size of a valid input has no fixed upper limit. For any particular run, the machine uses only finitely many cells. If it needs more, the model permits more tape. This idealization separates computability from the accidental memory limit of a physical computer.

2.3 Finite-state control

The machine has only *finitely many* states. This resembles human mechanical work: the brain keeps limited control information in mind, while details are recorded externally on paper. In

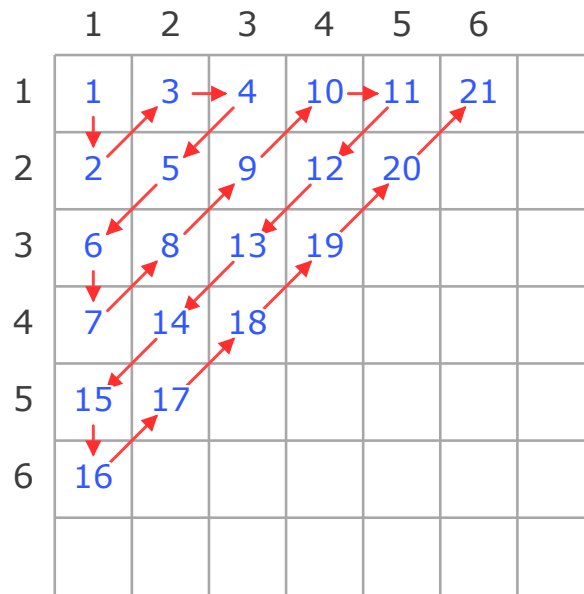


Figure 4: Mapping a two-dimensional grid to a one-dimensional tape.

a Turing machine, the states provide limited internal control and the tape provides external memory.

LEGO offers a similar analogy. There are finitely many types of pieces, yet arbitrarily many copies can be combined to build models without a fixed size limit. Likewise, a Turing machine reuses finitely many states and rules across arbitrarily many tape cells.

Finite control is required for two reasons. First, a mechanical controller must have a finite description and use limited physical resources. Second, infinitely many states could store one answer for every possible input. The machine would then be an infinite lookup table or enumeration, not an algorithm applying reusable steps. Finite states prevent this shortcut and require genuine computation by a finite set of rules.

2.4 Unbounded execution and halting

Unbounded execution means that the model imposes no fixed upper limit on the number of steps a computation may take. It does not mean that every computation runs forever. A particular run may halt after finitely many steps, but different inputs may require increasingly many steps, and some runs may never halt.

Unbounded execution is different from unbounded memory. A machine can execute forever using only a fixed amount of tape by repeatedly visiting the same cells in a loop. Conversely, the unbounded tape means that no fixed memory limit is imposed in advance, although only finitely many cells have been visited at any finite point in a run. Time counts transitions; memory counts the tape cells used.

Halting

A computation *halts* when the machine reaches a designated halting state or when no transition rule applies. This event is its *halting condition*. If the condition is never reached, the machine continues executing indefinitely.

For algorithms computing a function, a correct algorithm must halt on every input covered by

its specification and produce the required output. Reactive systems such as network services, by contrast, are specified to run indefinitely; their correctness is defined by safety and liveness properties rather than eventual termination.

2.5 A Turing-machine view of long division

Fix the divisor at 113. A state can record the current remainder r , where $0 \leq r < 113$. When the head reads the next decimal digit d , compute

$$t = 10r + d, \quad c = \left\lfloor \frac{t}{113} \right\rfloor, \quad r' = t \bmod 113.$$

The machine writes the quotient digit c to the tape, moves right, and changes to the state representing r' . The remainder can be kept in the finite control because it is always one of the 113 values from 0 to 112. The quotient is different: it can have nearly as many digits as the dividend and therefore has no fixed maximum length. A finite set of states cannot store an arbitrarily long quotient, so the machine must write its digits incrementally to the tape. For the input 35500001, the computation is:

Digit read	Quotient prefix	Remainder state
3	0	3
5	00	35
5	003	16
0	0031	47
0	00314	18
0	003141	67
0	0031415	105
1	00314159	34

After the machine reaches the blank cell, it halts. The final non-leading-zero quotient is 314159 and the remainder state is 34.

What the model establishes

The Turing machine is not a blueprint for a fast computer. It is a deliberately minimal mathematical model that makes the phrase “mechanical procedure” precise enough to study.

3 More Algorithms and More Machines

3.1 The Euclidean algorithm

Another example problem is to find the *greatest common divisor*: the largest positive integer that divides both given integers without a remainder. In this example, we want to find the greatest common divisor of 404 and 96, written $\text{gcd}(404, 96)$.

To compute it, repeatedly replace the larger pair by the divisor and the remainder:

$$\begin{aligned} 404 &= 4 \cdot 96 + 20, \\ 96 &= 4 \cdot 20 + 16, \\ 20 &= 1 \cdot 16 + 4, \\ 16 &= 4 \cdot 4 + 0. \end{aligned}$$

The last nonzero remainder is 4, so $\text{gcd}(404, 96) = 4$.

Like long division, this procedure uses a fixed collection of operations and a repeat rule. We can therefore design another Turing machine whose states implement it.

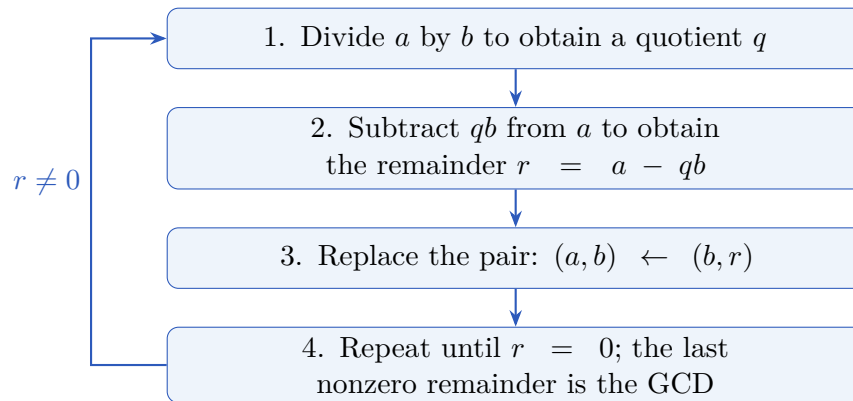


Figure 5: The four-step Euclidean algorithm for computing a greatest common divisor.

The Euclidean algorithm can be implemented directly as follows:

```
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a
```

3.2 The prime-factorization method

Prime factorization provides another method for finding a greatest common divisor. Factor each input into primes, retain every prime shared by both factorizations, and use the smaller exponent when a shared prime occurs more than once. For the same example,

$$404 = 2^2 \cdot 101, \quad 96 = 2^5 \cdot 3.$$

The only shared prime is 2, and the smaller exponent is 2. Therefore,

$$\text{gcd}(404, 96) = 2^2 = 4.$$

This method computes the same GCD function as the Euclidean algorithm, but it requires prime factorizations and therefore has a very different computational cost.

Homework

Our course code is 44,511,105. Find its prime factorization.

3.3 Functions versus algorithms

Function and algorithm

The *problem specification* states what output is required for each valid input. When every input has a unique output, this input–output relationship is a *function*; a search problem may instead permit several valid outputs. An *algorithm* is a particular sequence of instructions for computing an output that satisfies the specification. Thus, a function specifies *what* must be computed, whereas an algorithm specifies *how* to compute it. Several different algorithms may compute the same function or solve the same problem.

For example, $\text{gcd}(a, b)$ is a function. The Euclidean algorithm and a factorization-based method are algorithms that compute it.

Integer multiplication is also a function. The schoolbook multiplication algorithm for two n -digit integers uses $O(n^2)$ single-digit multiplications, while faster multiplication algorithms compute the same function with a lower asymptotic running time.

4 The Universal Turing Machine

Building a separate physical machine for every algorithm would be possible in principle but impractical. A universal Turing machine U accepts both an encoded machine $\langle M \rangle$ and an input x :

$$U(\langle M \rangle, x) = M(x).$$

It reads the description of M and simulates M step by step on x . Thus, one fixed machine can perform any computation described by another Turing machine.

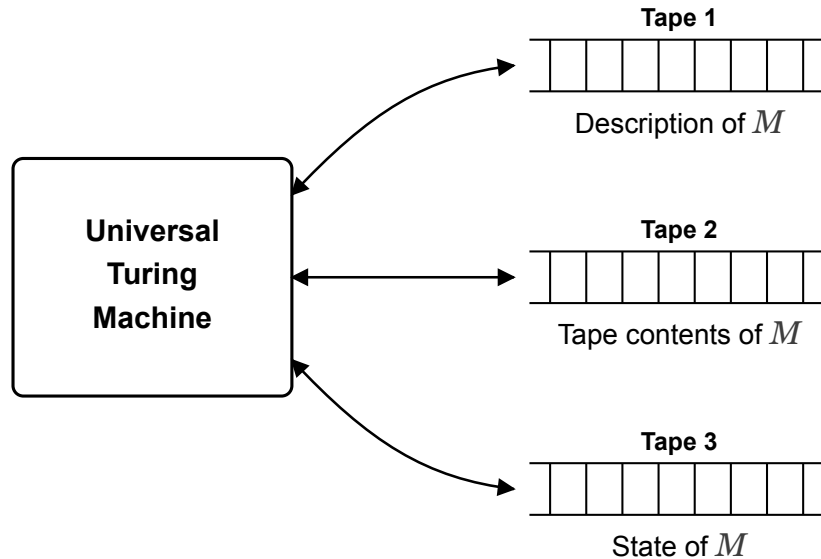


Figure 6: A three-tape organization of a universal Turing machine.

In this organization, Tape 1 stores the encoded description of M , Tape 2 stores the simulated contents of M 's tape, and Tape 3 records M 's current state. The universal machine repeatedly consults the description on Tape 1 and updates Tapes 2 and 3 exactly as M would, thereby reproducing M 's computation one transition at a time.

The key idea is that a program and ordinary data can both be represented as symbols. A program can therefore be stored, copied, inspected, interpreted, compiled, or generated like other data. This *program-as-data* principle underlies stored-program computers, operating systems, compilers, interpreters, program analyzers, and coding tools.

5 The Church–Turing Thesis

Church–Turing thesis

Every function that is effectively computable by an algorithm can be computed by a Turing machine.

Here, *effectively computable* means computable by a mechanical, rule-following procedure. The thesis identifies this informal idea with the mathematically defined Turing-machine model.

It is a *thesis*, not a theorem, because “effectively computable” is an informal concept rather than a separately formalized mathematical definition. No proof can connect a formal model to an informal notion in the ordinary theorem-proof sense. The thesis receives strong support from the fact that many independently proposed models, including lambda calculus, register machines, general-purpose programming languages, and idealized computers with unbounded memory, have the same computational power through mutual simulation.

A system is called *Turing complete* when it can simulate a universal Turing machine, subject to the same idealization of unbounded memory and execution time. Finite control, memory that can grow without a fixed bound, and conditional looping provide the essential ingredients.

5.1 What would challenge the thesis?

A genuine counterexample would require an effective, physically realizable procedure that computes a function no Turing machine can compute. Proposals involving infinite precision or exotic space-time models challenge the physical assumptions. Ordinary quantum computers do not currently provide such a counterexample: they may change efficiency, but standard quantum computation remains within the class of Turing-computable functions.

6 Limits of Computation

The Church–Turing thesis characterizes the power of mechanical computation, but it does not say that every clearly stated problem has an algorithmic solution.

6.1 Computable numbers

A real number is computable if a finite program can generate its digits to any requested precision. There are only countably many finite programs but uncountably many real numbers. Therefore, most real numbers are not computable, even though familiar constants such as π and e are computable.

6.2 The halting problem

Suppose a universal procedure $H(P, x)$ could always determine whether program P eventually halts on input x . Construct a program D that uses H on its own description:

```
def D():  
    if H(D, D):  
        loop_forever()  
    else:  
        return
```

If H predicts that D halts, D loops. If H predicts that D loops, D halts. Either answer contradicts the prediction. Therefore, no such universal halting decider exists.

This result is a consequence of the same program-as-data principle that makes universal computation possible. A sufficiently general computer can receive descriptions of programs, including descriptions that refer to themselves.

The price of universality

Universal computers can simulate arbitrary programs, but no universal program can perfectly predict every semantic property of arbitrary programs. Rice's theorem generalizes this limitation: every nontrivial semantic property of the function computed by an arbitrary program is undecidable.

7 Stored-Program Computers

In a stored-program computer, memory contains both instructions and data. A CPU repeatedly performs a fetch–decode–execute cycle:

1. fetch the instruction whose address is in the program counter;
2. decode the instruction;
3. execute it and update registers or memory;
4. update the program counter.

Conditional branches and jumps change the program counter, making selection and repetition possible.

The von Neumann architecture is a prominent organization of this idea: a processor communicates with a shared memory that stores both code and data. It is a practical architecture, while a Turing machine is a mathematical model. Both embody finite control operating on expandable memory.

Blackboard illustration placeholder

Von Neumann architecture: CPU with control unit, ALU, registers and program counter; shared memory containing code and data; input/output connections.

Replace this box with the classroom image.

8 Complexity: The Cost of Computation

Computability asks whether some algorithm exists. Complexity asks how resources grow with input size. The most common resources are running time and memory.

8.1 Input size and asymptotic growth

For an integer, input size usually means the number of digits or bits, not the integer's numerical value. If a decimal integer has n digits, a digit-by-digit algorithm naturally performs work as a function of n .

Big- O notation describes an asymptotic upper bound. It suppresses constant factors, lower-order terms, exact processor speed, and behavior on small inputs. Common growth rates include

$$O(1), \quad O(\log n), \quad O(n), \quad O(n \log n), \quad O(n^2), \quad O(2^n).$$

For sufficiently large n , the shape of the growth function matters more than small constant differences.

8.2 Searching an attendance list

Finding a name in an unsorted list by checking entries in order takes $O(n)$ time. If the list is sorted, binary search takes $O(\log n)$ comparisons: about 7 comparisons for roughly 100 entries and about 30 for one billion entries. A hash table can provide $O(1)$ expected lookup time by using additional memory and preprocessing.

This example illustrates a recurring time–space tradeoff: a method may run faster because it stores a more helpful representation of the data.

8.3 Fibonacci numbers

The recurrence

$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2)$$

does not determine a unique algorithm.

Method	Time	Extra space	Main idea
Naive recursion	$O(2^n)$	$O(n)$	Recomputes the same subproblems
Dynamic programming	$O(n)$	$O(n)$	Stores all previously computed values
Iterative	$O(n)$	$O(1)$	Keeps only the two latest values

Blackboard illustration placeholder

Recursion tree for naive Fibonacci, highlighting repeated calls such as $F(n-2)$.

Replace this box with the classroom image.

8.4 P and NP**P and NP**

P is the class of decision problems solvable in polynomial time. **NP** is the class of decision problems for which a proposed yes-certificate can be verified in polynomial time.

Every problem in P is in NP because a problem that can be solved efficiently can also have its answer verified efficiently. Whether every NP problem is also in P remains unknown:

$$P \stackrel{?}{=} NP.$$

Subset Sum illustrates the distinction. Given integers and a target, the decision problem asks whether some subset sums to the target. A proposed subset is easy to check, but naive search examines as many as 2^n subsets. This does not prove that the problem lacks a polynomial-time algorithm; it explains why verification may appear much easier than discovery.

9 Hierarchical Abstraction

The Turing machine tells us what a mechanical computer can compute. Computer architecture explains how a physical machine executes instructions. Abstraction explains how people build usable systems on top of that machinery.

Abstraction and interface

An *abstraction* hides details that are unnecessary for the current task. An *interface* specifies what a component makes available to its users without requiring them to know its internal implementation.

When writing `2 + 3` in Python, a programmer does not need to reason about transistors. The operation passes through layers such as language semantics, interpreter or compiler, machine instructions, digital circuits, logic gates, transistors, and electrical behavior. Each layer exposes a simpler interface to the layer above it.

Likewise, clicking *Send* in a messaging application may involve the application interface, operating-system services, network protocols, device drivers, hardware, and physical signals. No single application developer needs to rebuild all of these layers.

Blackboard illustration placeholder

Hierarchical abstraction stack, for example: user → application → operating system
→ network or instruction-set layer → hardware → physical signals.

Replace this box with the classroom image.

9.1 Leaky abstractions

An abstraction leaks when lower-level details affect behavior visible at a higher level. Examples include:

- a Python list appears able to grow, but physical memory remains finite;
- a network request resembles a function call, but latency and failure remain possible;
- a file appears to be a stable sequence of bytes, but storage devices can fail;
- two algorithms implement the same function, but their running times may differ dramatically.

Abstraction does not erase lower layers. It allows us to ignore them until performance, correctness, security, or failure forces us to cross an interface and inspect the implementation.

10 Concept Checklist

You should be able to explain each pair or relationship

- input, output, instruction, algorithm, and function;
- decision, search, counting, and optimization problems;
- model, simulation, mechanical procedure, and computability;
- tape, state, transition function, and halting state;
- Turing machine, universal Turing machine, and Turing completeness;
- stored-program computer, von Neumann architecture, and program counter;
- thesis versus theorem, and the Church–Turing thesis;
- computability versus decidability, including the halting problem;
- input size, Big- O , time complexity, and space complexity;
- recursion, dynamic programming, and iterative computation;
- P, NP, efficient solution, and efficient verification;
- abstraction, interface, implementation, and leaky abstraction.

11 Study Questions

1. Why does the long-division example qualify as a mechanical procedure?
2. What changes when divisibility is posed as a decision problem rather than a remainder-search problem?
3. Why does a Turing machine use an unbounded tape but only finitely many control states?
4. Using the state-update rule $t = 10r + d$, reproduce the remainder sequence for $35,500,001/113$.
5. Explain why the Euclidean algorithm and prime factorization may compute the same GCD function while having different costs.
6. What information must a universal Turing machine receive to simulate another machine?
7. Why is the Church–Turing statement called a thesis rather than a theorem?
8. How does program-as-data enable both stored-program computing and the halting-problem proof?
9. Compare linear search, binary search, and hash lookup. What assumptions and tradeoffs distinguish them?
10. Why do naive recursion and iteration have different costs for Fibonacci numbers even though both compute $F(n)$?
11. State the P versus NP question without saying that NP means “non-polynomial.”
12. Give one example of an abstraction leak and identify the lower-level detail that becomes visible.

Homework

Watch the CS50 Week 0 video and complete all assigned parts:

<https://cs50.harvard.edu/x/weeks/0/>

Sources Used for These Notes

These notes synthesize the slide deck *01 Computation and Abstraction.pptx*, its embedded speaker notes, and the two classroom transcripts dated 8 September 2026. The wording has been edited for clarity, and transcription errors have been corrected using the mathematical context. Blackboard drawings that were described in speech but unavailable as images remain as labeled replacement boxes.

For historical context on the Turing machine, see A. M. Turing, “On Computable Numbers, with an Application to the Entscheidungsproblem,” *Proceedings of the London Mathematical Society*, 1936. For a detailed discussion of the Church–Turing thesis, see <https://plato.stanford.edu/entries/church-turing/>.